NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Artificial Intelligence

Lecture 4: Adversarial Search (Demo version)

Adversarial Search

	O	X
O	X	X
O		X

	O	X
O	X	X
O		X

Minimax

O	X	X
O	O	
O	X	X

-1

X	O	X
O	O	X
X	X	O

0

O		X
	X	O
X	O	X

1

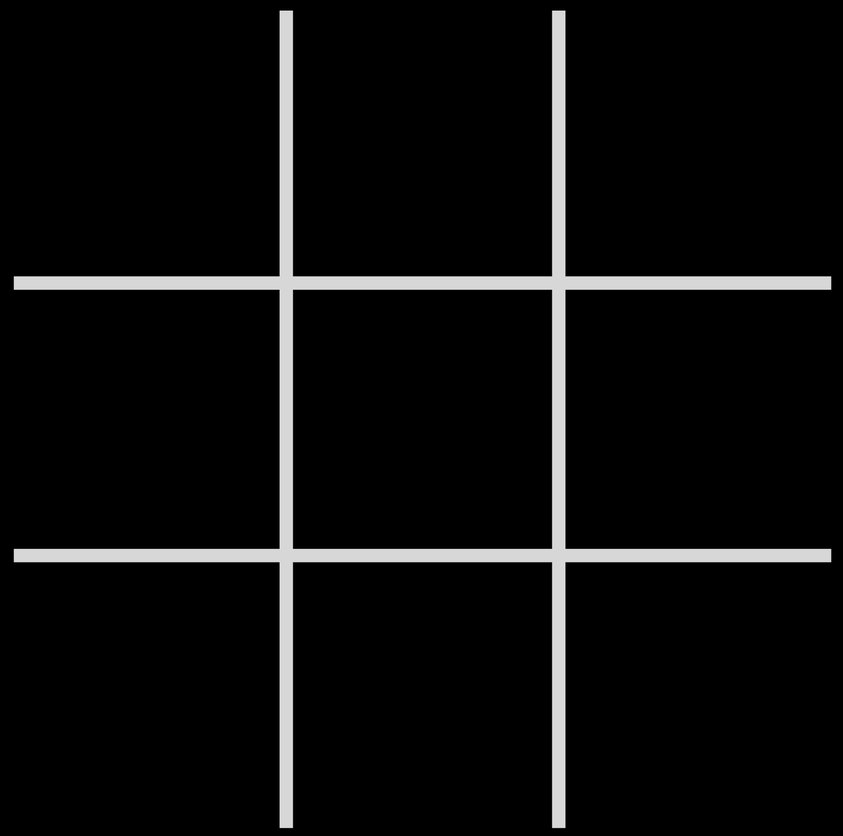
Minimax

- MAX (X) aims to maximize score.
- MIN (O) aims to minimize score.

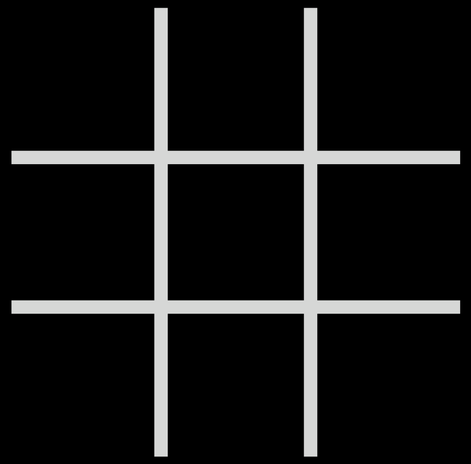
Game

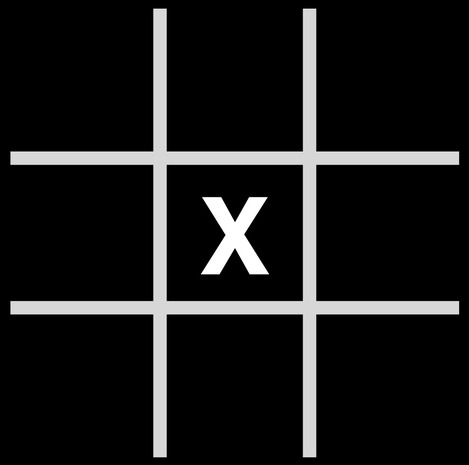
- S_0 : initial state
- $\textsc{Player}(s)$: returns which player to move in state s
- $\textsc{Actions}(s)$: returns legal moves in state s
- $\textsc{Result}(s, a)$: returns state after action a taken in state s
- $\textsc{Terminal}(s)$: checks if state s is a terminal state
- $\textsc{Utility}(s)$: final numerical value for terminal state s

Initial State

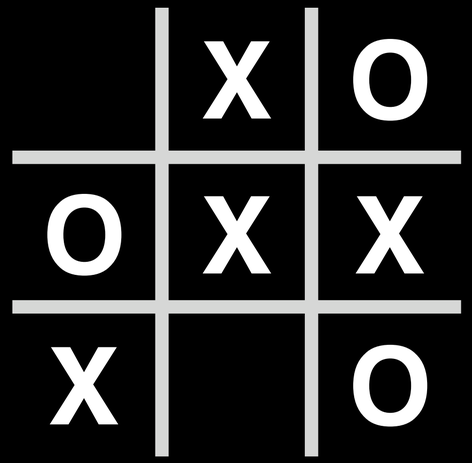


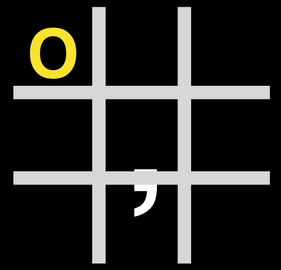
PLAYER(s)

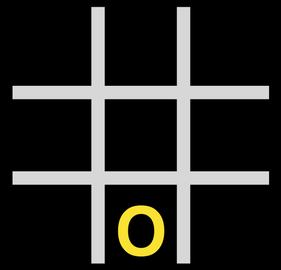
PLAYER() = X

PLAYER() = O

ACTIONS(s)

ACTIONS(

) = { 

, 

}

RESULT(s, a)

RESULT(| | X | O |
|---|---|---|
| O | X | X |
| X | | O |

, | | | |
|---|---|---|
| O | | |
| | | |
| | | |

) = | O | X | O |
|---|---|---|
| O | X | X |
| X | | O |

TERMINAL(s)

TERMINAL(| | | | | | | | |) = false

O		
O	X	
X	O	X

TERMINAL(| | | | | | | | |) = **true**

O		X
O	X	
X	O	X

UTILITY(s)

UTILITY(| | | |
|---|---|---|
| O | | X |
| O | X | |
| X | O | X |
|) = **1** | | |

UTILITY(| | | |
|---|---|---|
| O | X | X |
| X | O | |
| O | X | O |
|) = **-1** | | |

O	X	O
O	X	X
X	X	O

VALUE: 1

PLAYER(s) = O

MIN-VALUE: 0

	X	O
O	X	X
X		O

MAX-VALUE: 1

O	X	O
O	X	X
X		O

VALUE: 1

O	X	O
O	X	X
X	X	O

MAX-VALUE: 0

	X	O
O	X	X
X	O	O

VALUE: 0

X	X	O
O	X	X
X	O	O

PLAYER(s) = O

MIN-VALUE: 0

	X	O
O	X	X
X		O

MAX-VALUE: 1

O	X	O
O	X	X
X		O

VALUE: 1

O	X	O
O	X	X
X	X	O

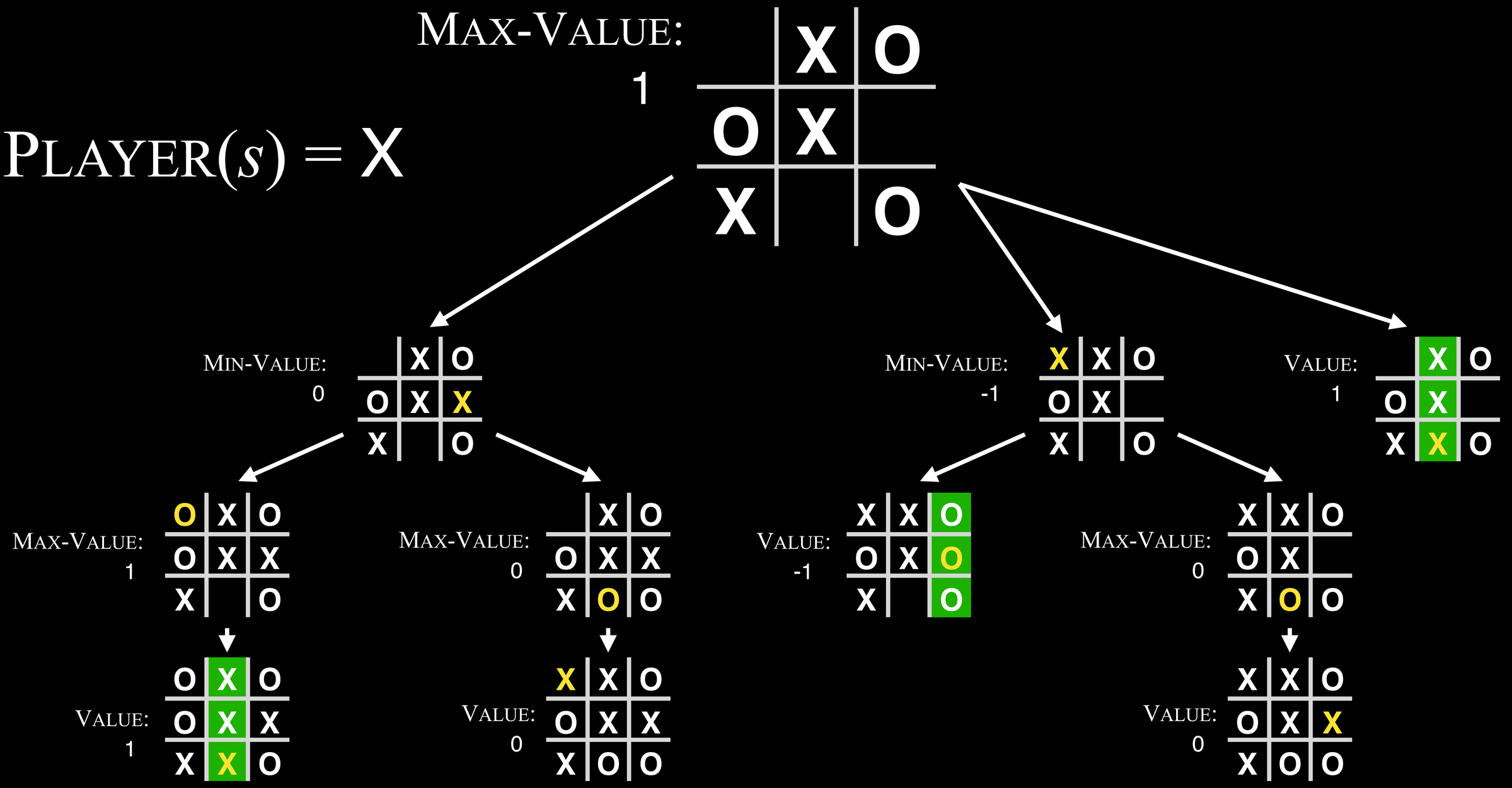
MAX-VALUE: 0

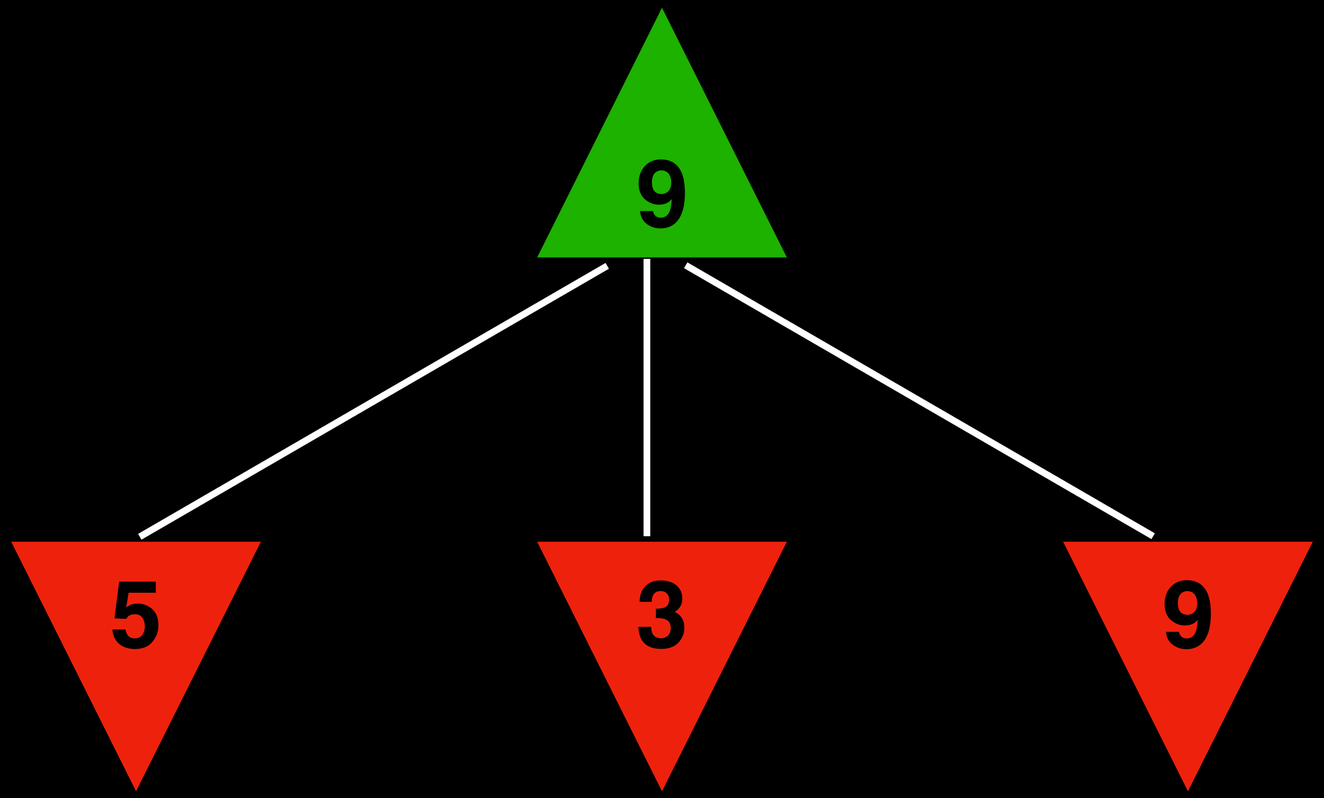
	X	O
O	X	X
X	O	O

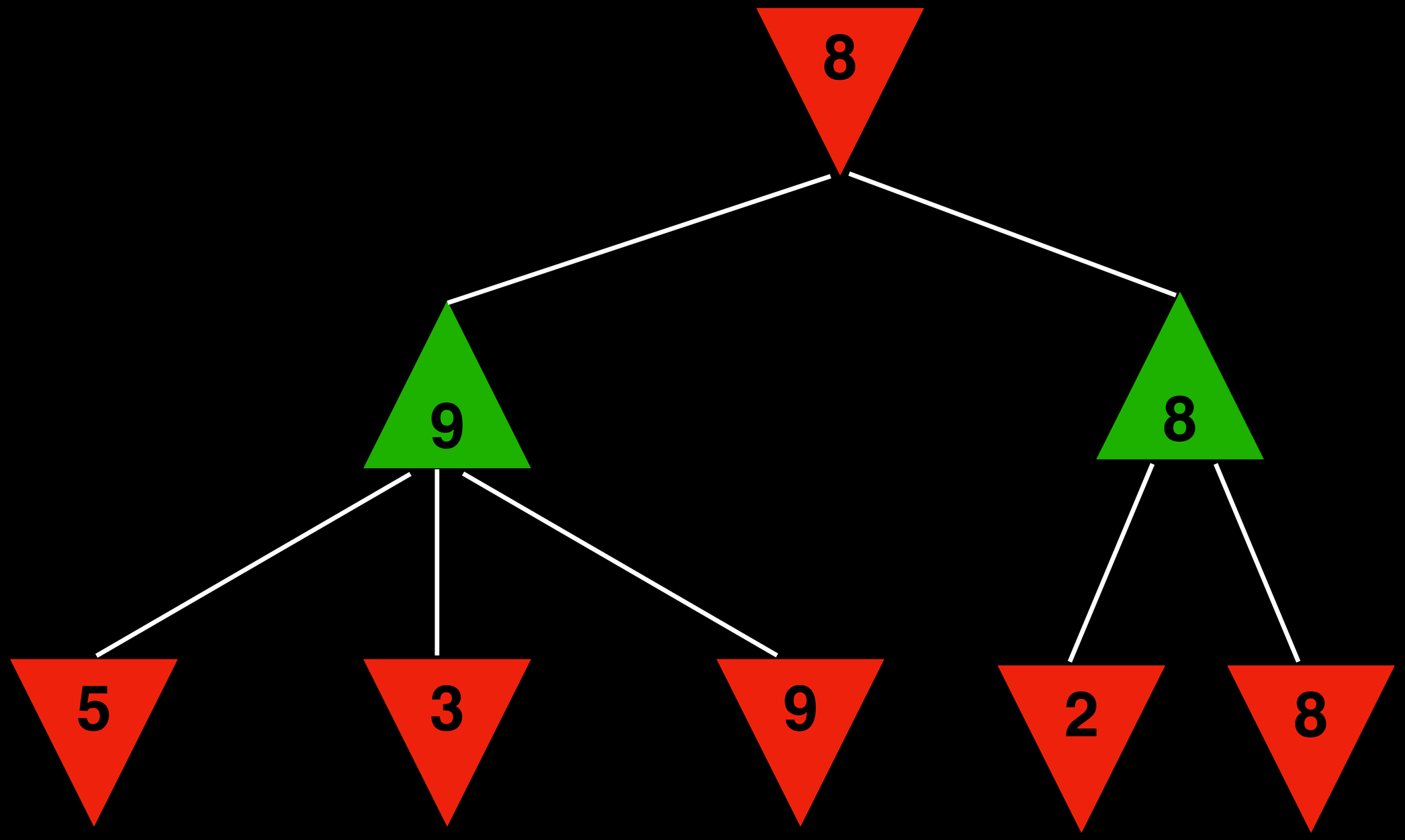
VALUE: 0

X	X	O
O	X	X
X	O	O

PLAYER(s) = X


MAX-VALUE: 1
MIN-VALUE: 0
MIN-VALUE: -1
VALUE: 1
MAX-VALUE: 1
MAX-VALUE: 0
VALUE: -1
MAX-VALUE: 0
VALUE: 1
VALUE: 0
VALUE: 0


9
5
3
9


8
9
8
5
3
9
2
8

Minimax

- Given a state s:
  - MAX picks action a in ACTIONS(s) that produces highest value of MIN-VALUE(RESULT(s, a))
  - MIN picks action a in ACTIONS(s) that produces smallest value of MAX-VALUE(RESULT(s, a))

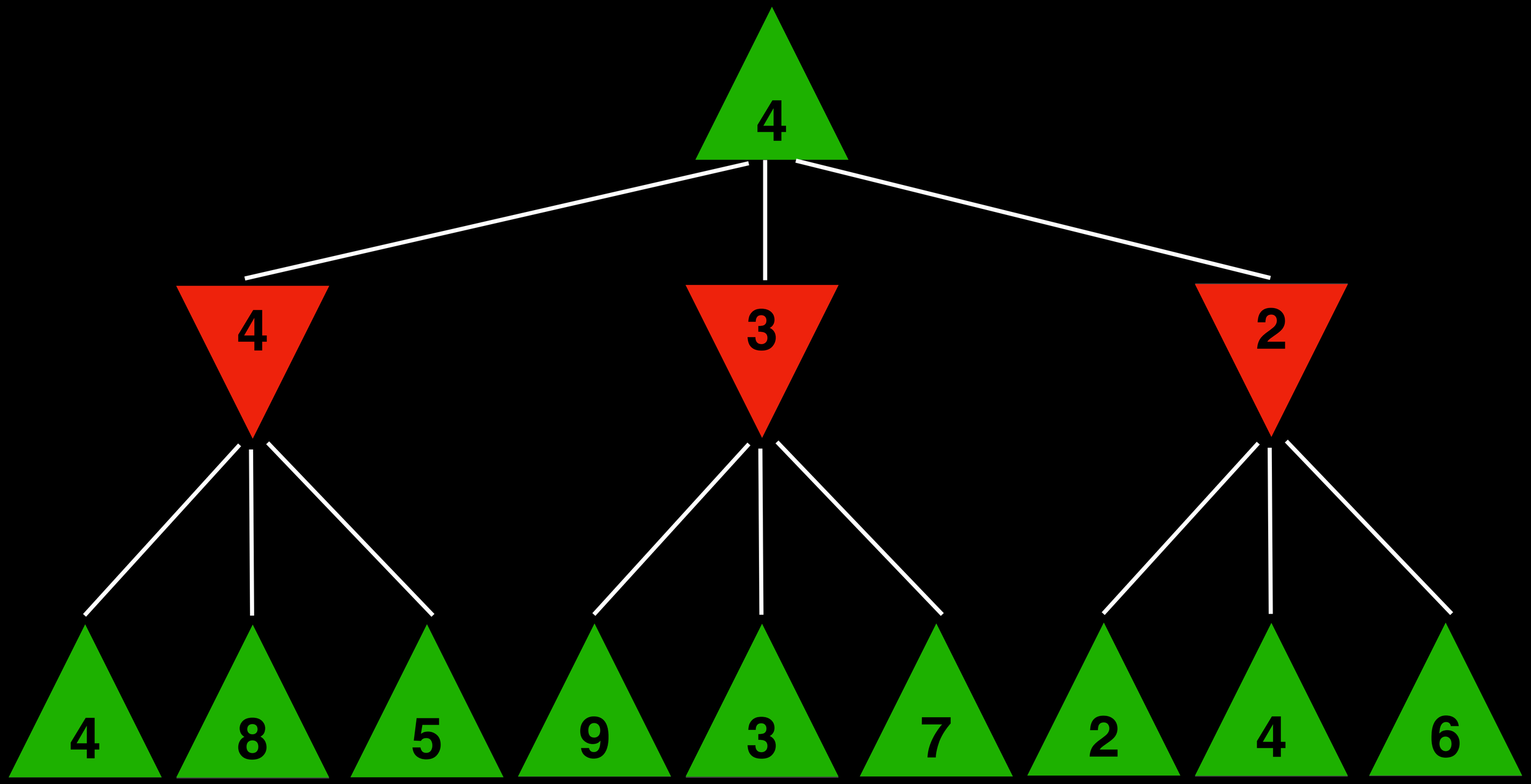
Minimax

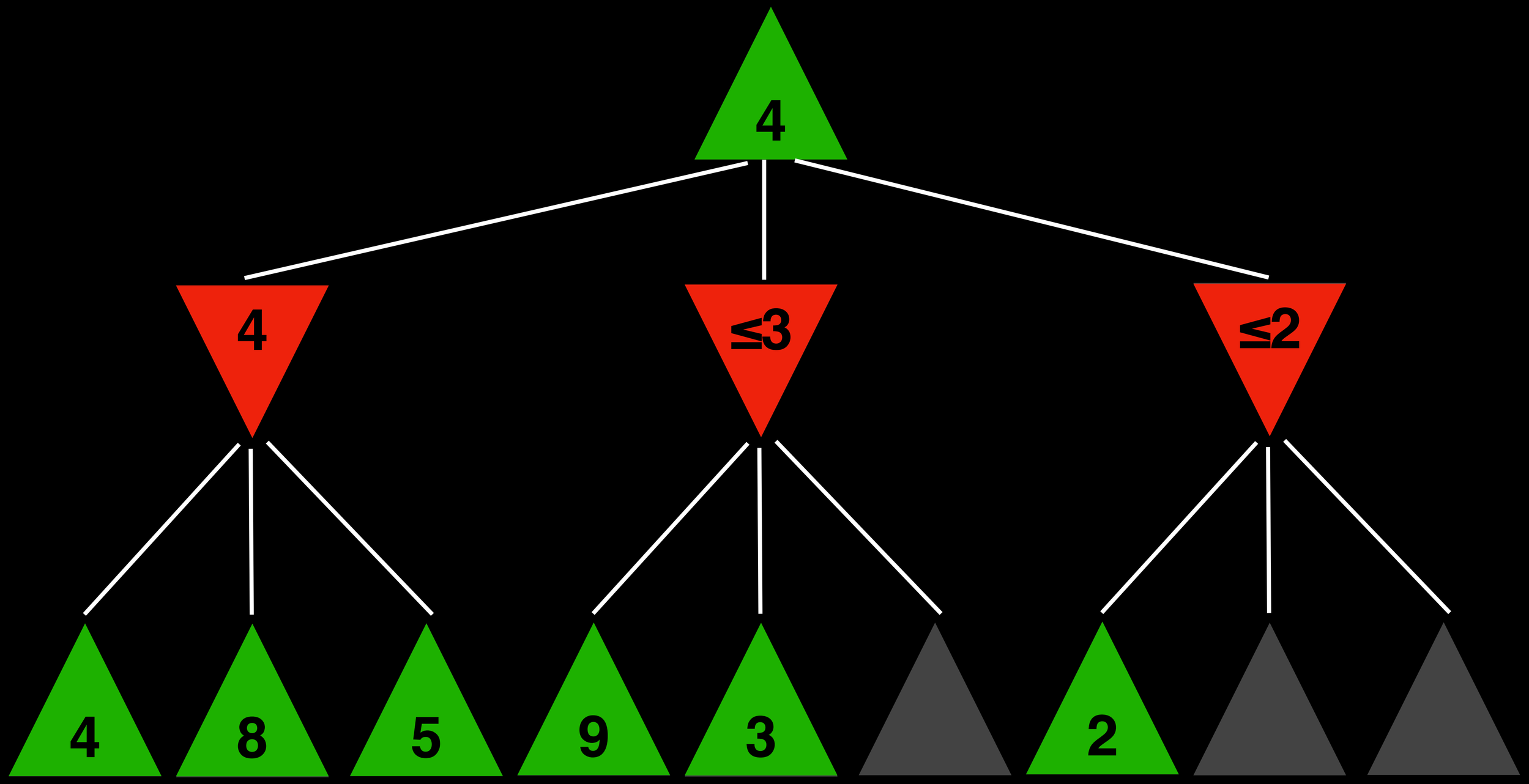
```
function MAX-VALUE(state):
    if TERMINAL(state):
        return UTILITY(state)
    v = -∞
    for action in ACTIONS(state):
        v = MAX(v, MIN-VALUE(RESULT(state, action)))
    return v
```

Minimax

```
function MIN-VALUE(state):
    if TERMINAL(state):
        return UTILITY(state)
    v = ∞
    for action in ACTIONS(state):
        v = MIN(v, MAX-VALUE(RESULT(state, action)))
    return v
```

Optimizations


4
4
3
2
4
8
5
9
3
7
2
4
6


4
4
≤3
≤2
4
8
5
9
3
2

Alpha-Beta Pruning

255,168

total possible Tic-Tac-Toe games

288,000,000,000

total possible chess games
after four moves each

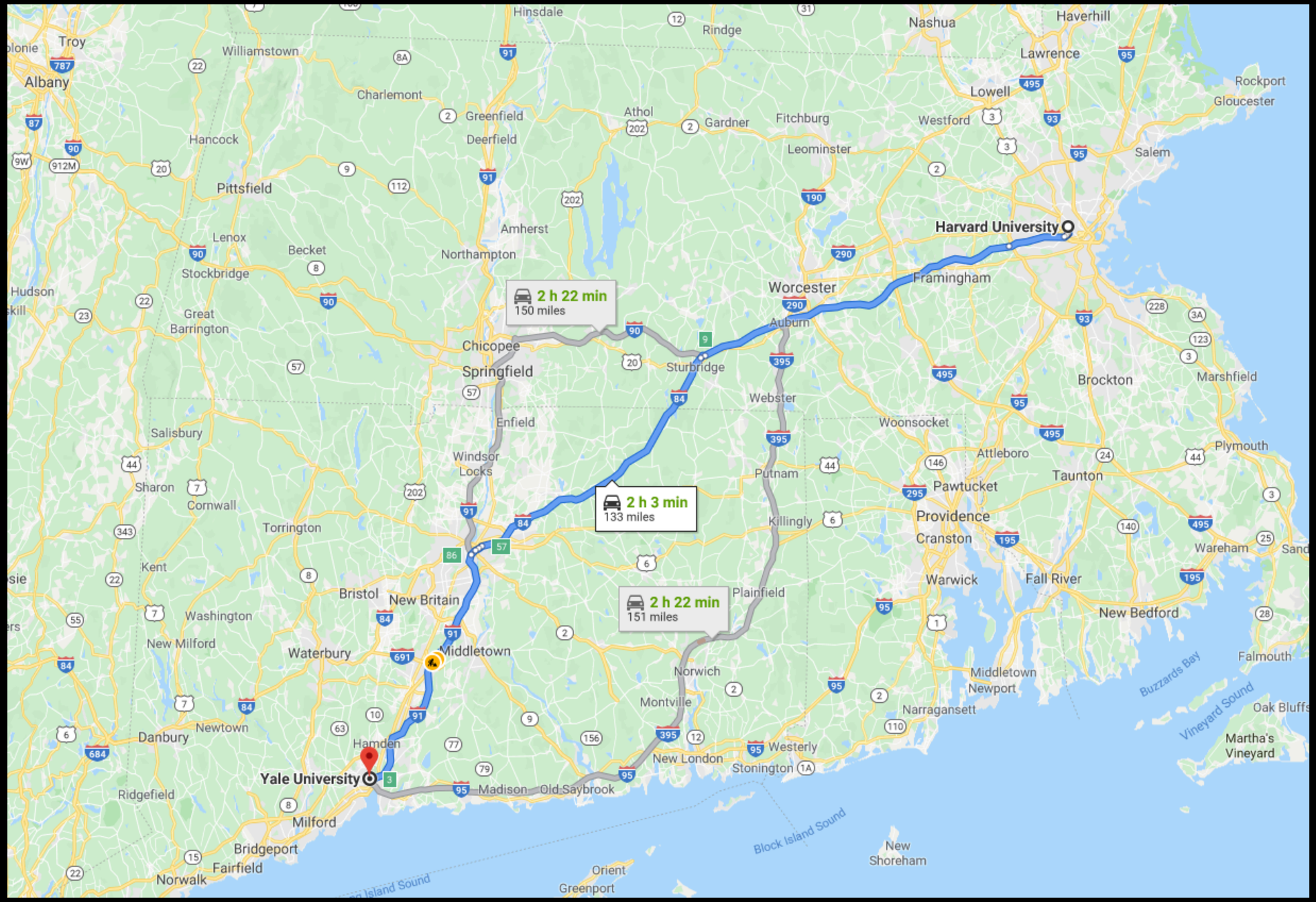
$$10^{29000}$$

total possible chess games
(lower bound)

Depth-Limited Minimax

evaluation function

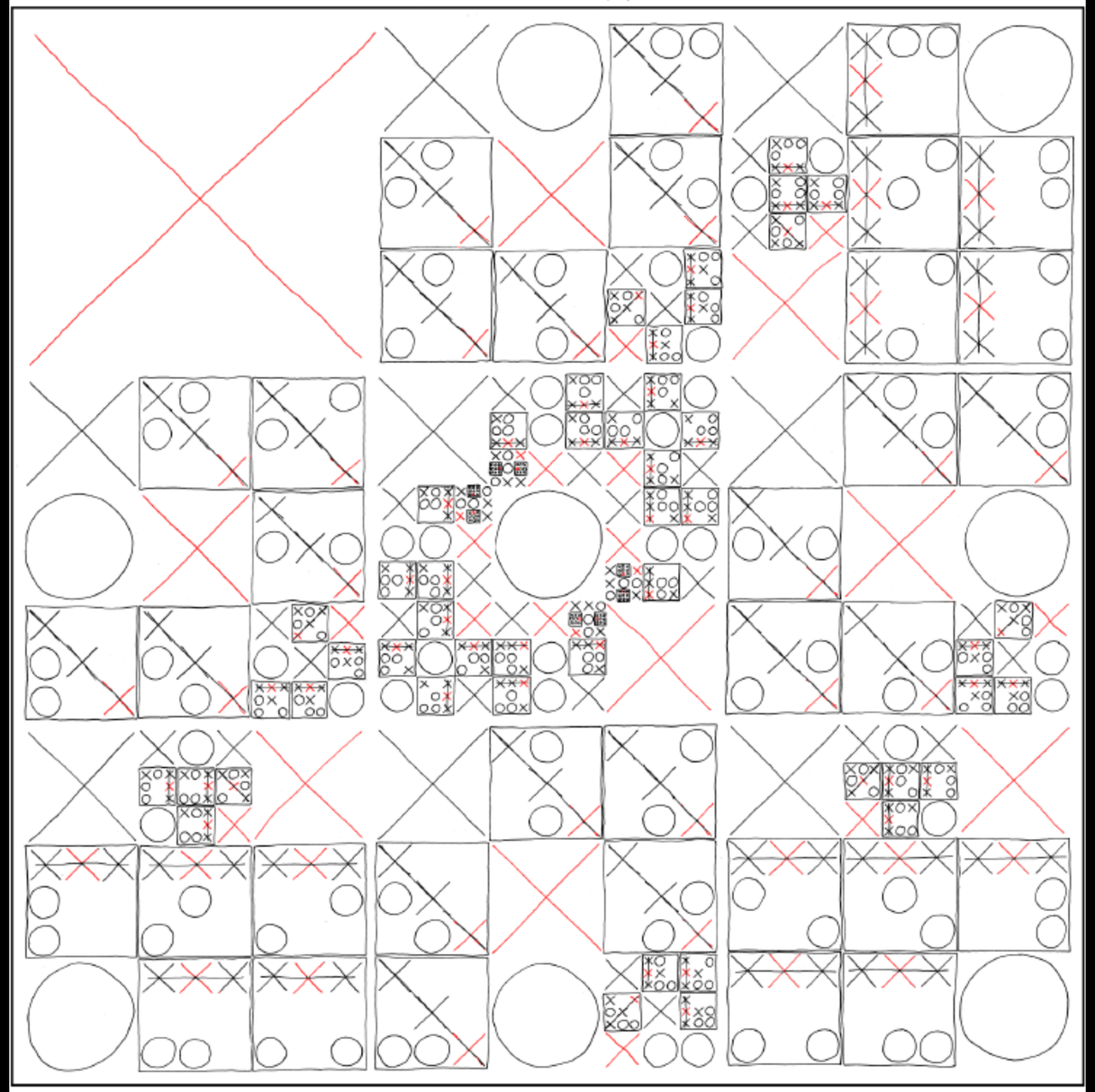
function that estimates the expected utility of the game from a given state


Harvard University
Yale University
2 h 22 min
150 miles
2 h 3 min
133 miles
2 h 22 min
151 miles
Hinsdale
Rindge
Nashua
Haverhill
Lawrence
Lowell
Rockport
Gloucester
Salem
Troy
Albany
Williamstown
Charlemont
Greenfield
Deerfield
Athol
Gardner
Fitchburg
Westford
Leominster
Hancock
Pittsfield
Lenox
Becket
Stockbridge
Amherst
Northampton
Hudson
Great Barrington
Worcester
Framingham
Auburn
Chicopee
Springfield
Sturbridge
Webster
Brockton
Marshfield
Enfield
Windsor Locks
Salisbury
Sharon
Cornwall
Torrington
Putnam
Woonsocket
Attleboro
Pawtucket
Taunton
Plymouth
Providence
Cranston
Killingly
Kent
Bristol
New Britain
Washington
New Milford
Waterbury
Middletown
Plainfield
Warwick
Fall River
New Bedford
Wareham
Falmouth
Buzzards Bay
Vineyard Sound
Oak Bluffs
Martha's Vineyard
Middletown
Newport
Narragansett
Norwich
Montville
Newtown
Danbury
Hamden
New London
Westerly
Stonington
Ridgefield
Madison
Old Saybrook
Milford
Bridgeport
Fairfield
Norwalk
Orient
Greenport
Block Island Sound
New Shoreham

COMPLETE MAP OF OPTIMAL TIC-TAC-TOE MOVES

YOUR MOVE IS GIVEN BY THE POSITION OF THE LARGEST RED SYMBOL ON THE GRID. WHEN YOUR OPPONENT PICKS A MOVE, ZOOM IN ON THE REGION OF THE GRID WHERE THEY WENT. REPEAT.

MAP FOR X:



https://xkcd.com/832/

Search